

Direct Addressing Table

1. Concept

Definition

Direct Addressing is a technique where the key itself is used as the index of an array.

If keys are small integers:

$$\text{Address of key } k = k$$

No hash function is required.

2. Working Principle

We create an array T such that:

$$T[k] = \text{element with key } k$$

If a key does not exist:

$$T[k] = \text{NIL}$$

Access is done using direct array indexing.

3. Example (Small Key Range)

Suppose we store roll numbers:

$$\{2, 5, 8\}$$

Maximum key = 8

So we create:

$$T[0 \dots 8]$$

Visual Representation

0	1	2	3	4	5	6	7	8
		Data			Data			Data

Only indices 2, 5, 8 store values. Others remain empty (NIL).

4. Problem Example (Large Key Range)

Keys:

$\{10, 500000000, 999999999\}$

Maximum key $\approx 10^9$

We must create:

$T[0 \dots 10^9]$

Major Issue

Only 3 elements stored But memory allocated for 10^9 entries!

This causes extreme memory waste.

5. Time Complexity

Performance

Insert = $O(1)$
Search = $O(1)$
Delete = $O(1)$

Reason:

Array indexing is constant time

6. Space Complexity

Let:

- U = Universe size (maximum possible key)
- n = Number of stored elements

Space Required:

$$\boxed{O(U)}$$

Not:

$$O(n)$$

If:

$$U \gg n$$

Then most memory is unused.

7. When Is It Useful?

- Small key range
- U close to n
- Frequency counting
- Counting sort
- Boolean presence arrays

8. Core Trade-Off

Important Insight

Direct Addressing provides:

Perfect $O(1)$ time

But requires:

Huge $O(U)$ space

This trade-off leads to the development of **Hashing**.

Comparison: Direct Addressing vs Hashing

Key Difference

Direct Addressing uses the key itself as the index. Hashing compresses a large key space into a smaller table.

Feature	Direct Addressing	Hashing
Index Computation	$T[k]$	$T[h(k)]$
Space Complexity	$O(U)$	$O(m)$
Handles Large Keys	No	Yes
Memory Efficient	No	Yes
Collision Handling	Not needed	Required

Where:

U = Universe size (max possible key)

m = Hash table size

9. Memory Usage Example (Actual Numbers)

Assume:

- Keys range from 0 to 10^9
- Each table entry stores an 8-byte pointer

- Only $n = 100$ elements are stored

Direct Addressing Memory Requirement

We must create:

$$10^9 + 1 \text{ slots}$$

Each slot = 8 bytes

Total memory:

$$10^9 \times 8 \text{ bytes}$$

$$= 8 \times 10^9 \text{ bytes}$$

$$= 8 \text{ GB}$$

But we are storing only 100 elements!

Memory used for actual data:

$$100 \times 8 = 800 \text{ bytes}$$

Huge waste.

Hashing Memory Requirement

Suppose we choose:

$$m = 200$$

Each slot = 8 bytes

Total memory:

$$200 \times 8 = 1600 \text{ bytes}$$

Much smaller than 8 GB.

Conclusion from Memory Example

Key Insight

Direct Addressing wastes memory when key range is large. Hashing compresses a large universe into a manageable table.

Direct Addressing is practical only when $U \approx n$

Otherwise:

Hashing is the better choice.

**Direct Addressing = Extremely Fast but Memory
Expensive**